

Application Specific Computer Architectures

ELL 7282

Assignment-1: Encryption, Decryption, and Key-Based Hashing

Group C

Report

Fall – 2025

Submitted By -

Amay Jain (2025EET2478)

Akshat Jain (2025EET2884)

Submitted To -

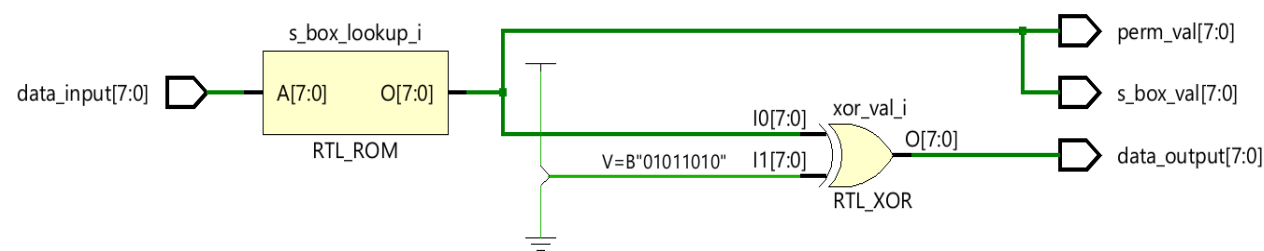
Prof. Kaushik Saha

Teaching Assistant –

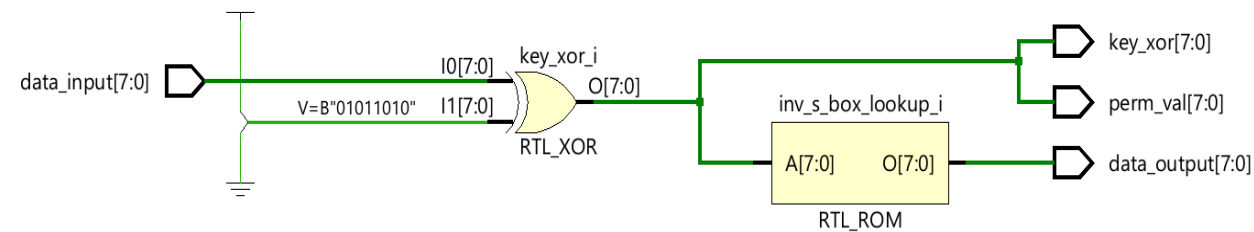
Garima Singhal

1. Detailed design description and architecture diagrams (block diagrams showing data flow and control).

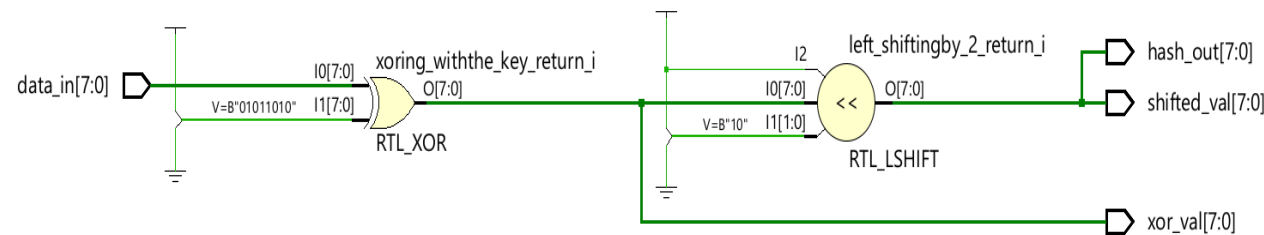
Encryption Block Diagram

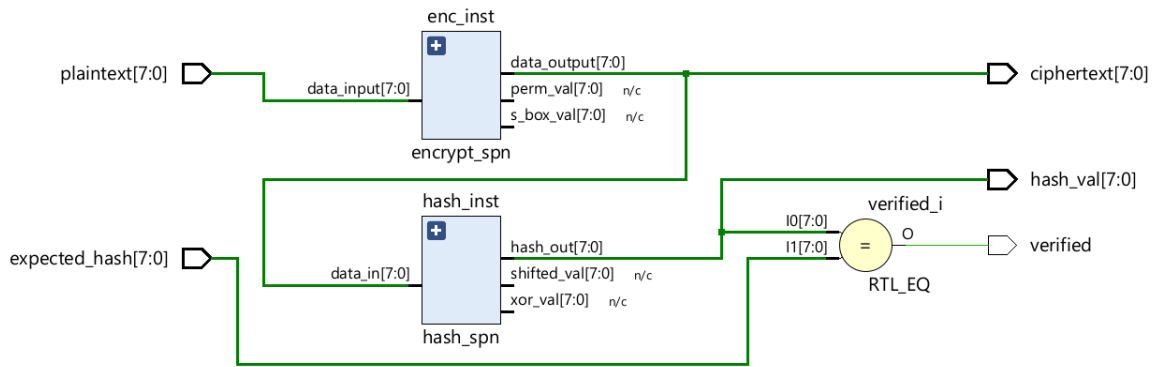


Decryption Block Diagram

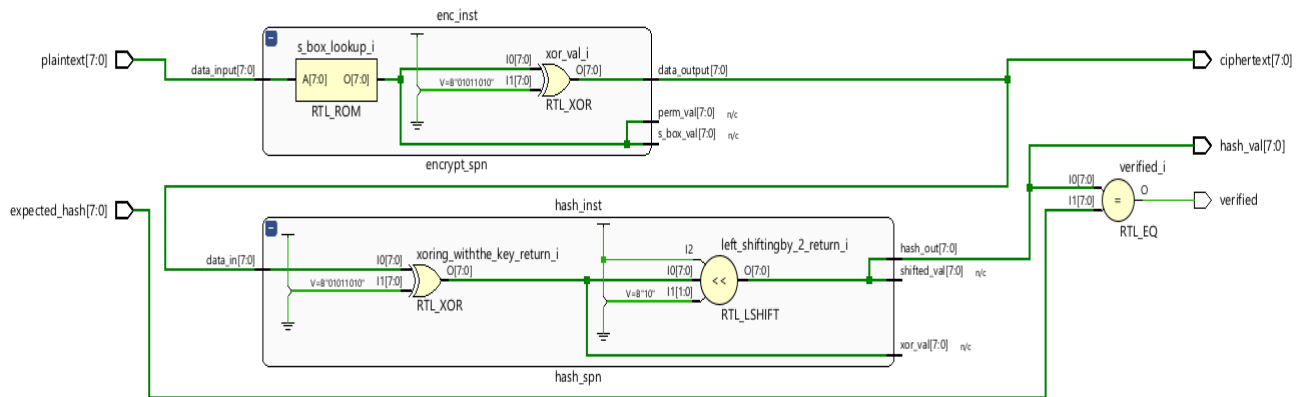


Hash Block Diagram





Verify Block Diagram



2. Implementation details (LUTs, combinational or pipelined logic)

The 8-bit SPN cipher was implemented targeting an FPGA using Verilog. The design focuses on simplicity and efficiency while ensuring correct encryption, decryption, hashing, and verification.

Logic Style

The design is primarily combinational:

S-box (Substitution): Implemented as a lookup table (LUT) using case statements.

P-box (Permutation): Implemented using bitwise wiring, which rearranges bits of the intermediate state.

Each encryption round processes the entire 8-bit plaintext combinationaly, with results available after one propagation delay.

Verification Logic

The verifying module compares the computed hash with the expected hash:

Implemented as a simple combinational equality check.

Sets the verified flag high if the hash matches.

All encryption rounds are combinational:

The encrypt and verifying modules process the input in one go using LUTs and bitwise wiring.

There are no intermediate registers between rounds, which would be required for pipelining.

Testbench waits only for propagation delay (#10):

We never clock the design multiple times or store intermediate states in registers, which is typical for pipelined designs.

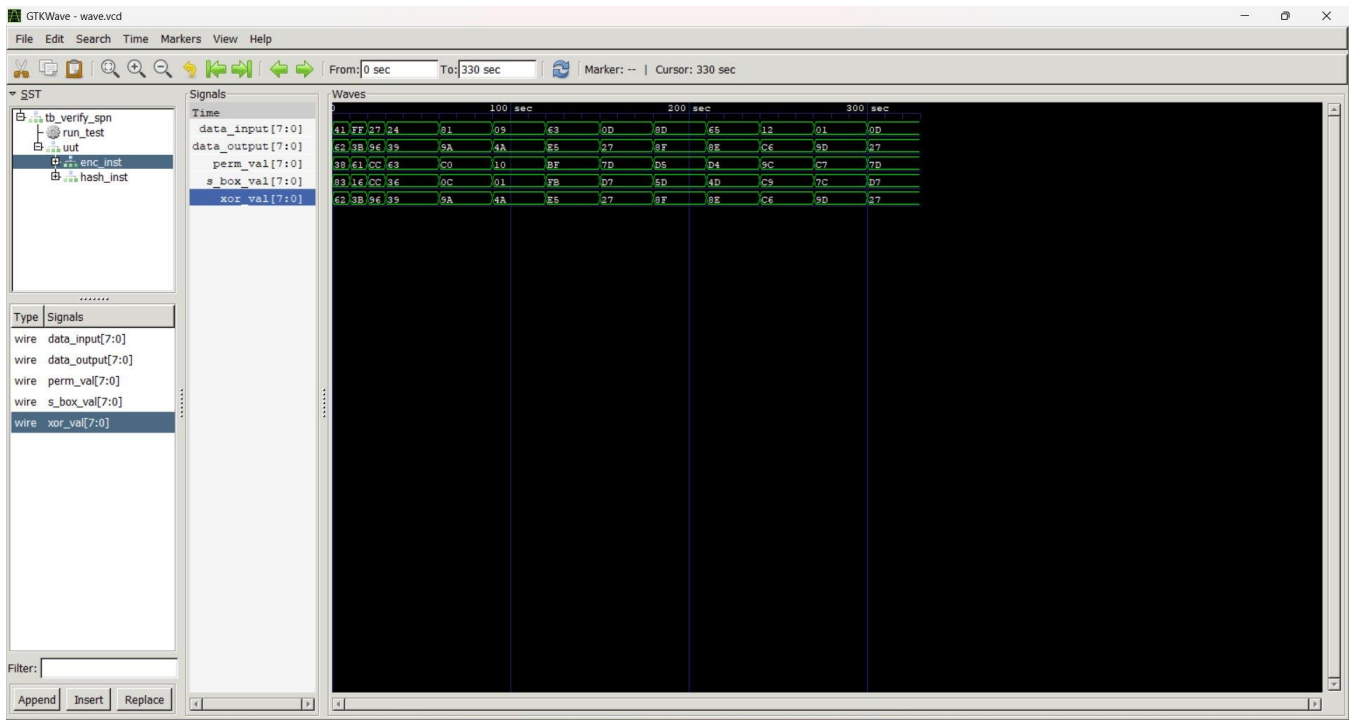
Verification logic is combinational:

The verified output is set directly from the comparison (`hash_val == expected_hash`), no registers or staged computation.

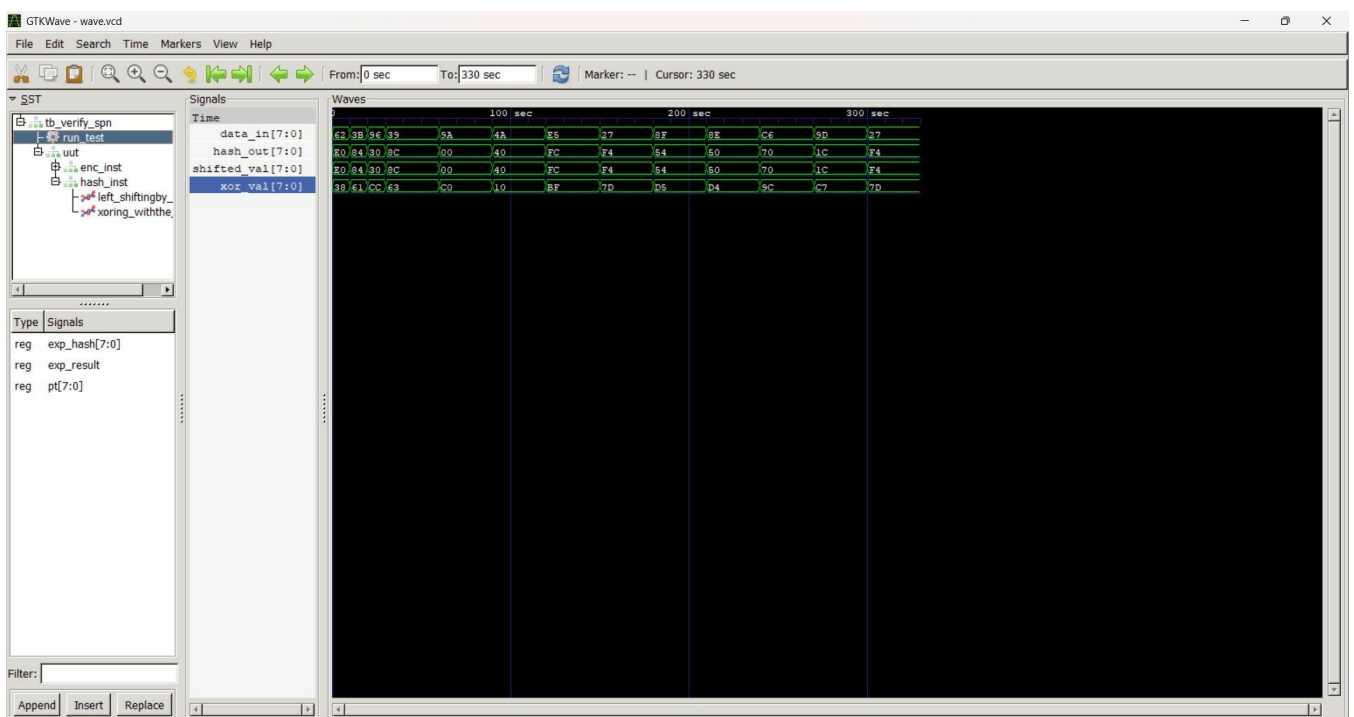
3. Simulation and synthesis results, including area, delay, and throughput

Simulation waveforms

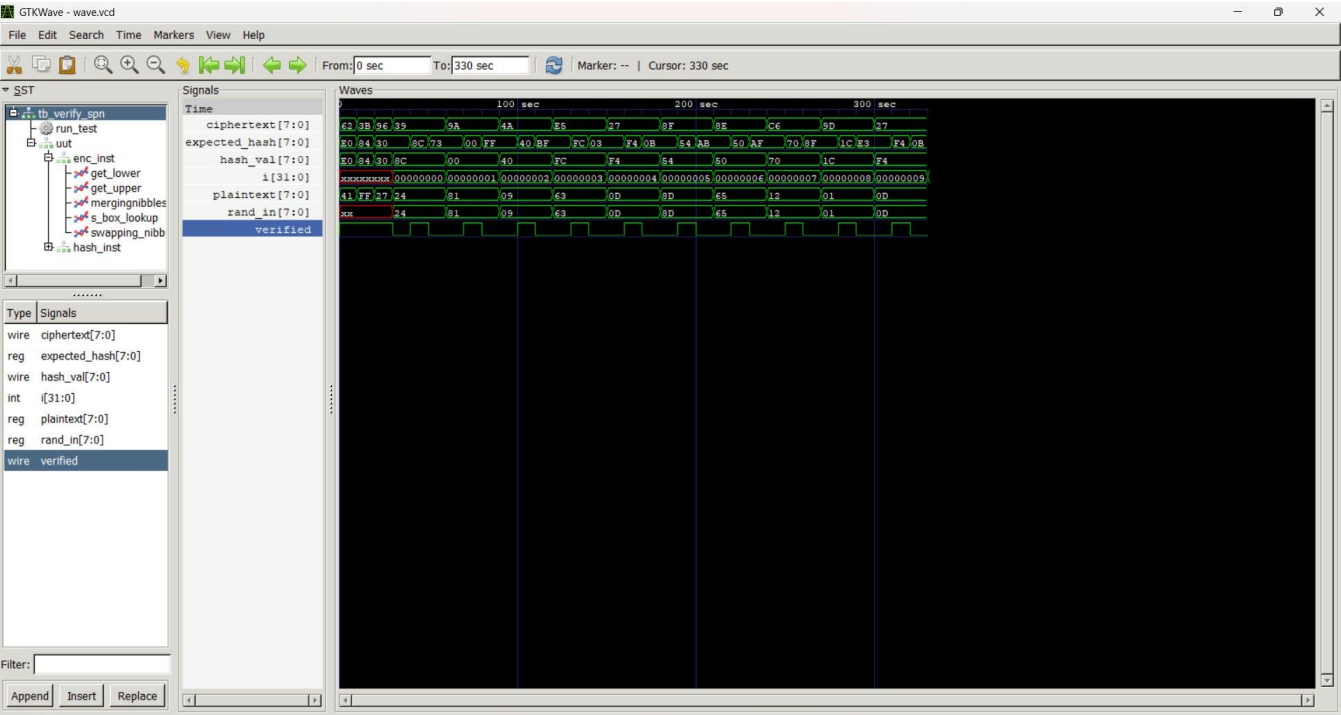
- Encryption



- Hash



- Verify



- Results

PLAINTEXT	CIPHERTEXT	HASH_VAL	EXPECTED_HASH	VERIFIED
41	62	e0	e0	1
ff	3b	84	84	1
27	96	30	30	1
-----HERE WE ARE RUNNING SOME RANDOM TEST CASES-----				
24	39	8c	8c	1
24	39	8c	73	0
81	9a	00	00	1
81	9a	00	ff	0
09	4a	40	40	1
09	4a	40	bf	0
63	e5	fc	fc	1
63	e5	fc	03	0
0d	27	f4	f4	1
0d	27	f4	0b	0
8d	8f	54	54	1
8d	8f	54	ab	0
65	8e	50	50	1
65	8e	50	af	0
12	c6	70	70	1
12	c6	70	8f	0
01	9d	1c	1c	1
01	9d	1c	e3	0
0d	27	f4	f4	1
0d	27	f4	0b	0

Synthesis

1. Area Report

Encryption

Report Cell Usage:

+-----+-----+-----+			
	Cell	Count	
+-----+-----+-----+			
1	LUT6		44
2	IBUF		8
3	OBUF		24

Decryption Area

Report Cell Usage:

+-----+-----+-----+			
	Cell	Count	
+-----+-----+-----+			
1	LUT1		4
2	LUT6		40
3	IBUF		8
4	OBUF		24
+-----+-----+-----+			

Hash Area

Report Cell Usage:

+-----+-----+-----+			
	Cell	Count	
+-----+-----+-----+			
1	LUT1		4
2	IBUF		8
3	OBUF		24
+-----+-----+-----+			

2. Critical Path Delay Report: It is the maximum Data Path Delay

Encryption: from data_input to data_output

Timing Report

Slack:inf

Source: data_input[5]

(input port clocked by virt_clk {rise@0.000ns fall@5.000ns period=10.000ns})

Destination: data_output[1]

(output port)

Path Group:(none)

Path Type:Max at Slow Process Corner

Data Path Delay:6.268ns (logic 3.756ns (59.919%) route 2.512ns (40.081%))

Logic Levels:4 (IBUF=1 LUT6=2 OBUF=1)

Input Delay:2.000ns

Clock Uncertainty:0.025ns ((TSJ^2 + TIJ^2)^1/2 + DJ) / 2 + PE

Total System Jitter(TSJ):0.050ns

Total Input Jitter(TIJ):0.000ns

Discrete Jitter(DJ):0.000ns

Phase Error(PE):0.000ns

Location	Delay type	Incr(ns)	Path(ns)	Netlist Resource(s)
(clock virt_clk rise edge)				
		0.000	0.000	r

Critical Path Delay: 6.268ns

Decryption, Hash, Verifying: from data_input to data_output

Timing Report

Slack:inf

Source: data_input[0]

(input port clocked by virt_clk {rise@0.000ns fall@5.000ns period=10.000ns})

Destination: data_output[0]

(output port)

Path Group:(none)

Path Type:Max at Slow Process Corner

Data Path Delay:6.257ns (logic 3.756ns (60.025%) route 2.501ns (39.975%))

Logic Levels:4 (IBUF=1 LUT6=2 OBUF=1)

Input Delay:2.000ns

Clock Uncertainty:0.025ns ((TSJ^2 + TIJ^2)^1/2 + DJ) / 2 + PE

Total System Jitter(TSJ):0.050ns

Total Input Jitter(TIJ):0.000ns

Discrete Jitter(DJ):0.000ns

Phase Error(PE):0.000ns

Location	Delay type	Incr(ns)	Path(ns)	Netlist Resource(s)
(clock virt_clk rise edge)				
		0.000	0.000	r

Critical Path Delay: 6.257ns

3. Throughput Report

Throughput = Bits per cycle * clock frequency

frequency = $1/\text{Critical Path Delay}$

Encryption:

Critical Path Delay = 6.268ns

frequency = $1/6.268\text{ns} = 159.54 \text{ MHz}$

Bits per Cycle = 8

Throughput = $8 * 159.54 \text{ M} = 1.276 \text{ Gbps}$

Decryption, Hash, Verifying:

Critical Path Delay = 6.527ns

frequency = $1/6.527\text{ns} = 153.21 \text{ MHz}$

Bits per Cycle = 8

Throughput = $8 * 153.21 \text{ M} = 1.226 \text{ Gbps}$

4. Verification methodology and testbench description.

1. Verification Objective

The primary goal of verification is to ensure that the 8-bit SPN cipher:

Correctly encrypts the input plaintext.

Generates the correct ciphertext and hash values.

Properly verifies whether the computed hash matches the expected hash.

Handles both fixed and random test vectors to confirm robustness.

2. Verification Approach

Functional Verification:

The design was verified using a Verilog testbench (testbench.v).

Test vectors are applied to the DUT, and outputs are compared against expected results.

Types of Test Cases

Fixed Test Cases:

Manually chosen input-output pairs to validate known behaviour.

Examples:

PLAINTEXT	EXPECTED HASH
-----------	---------------

0x41	0xE0
------	------

0xFF	0x84
------	------

0x27	0x30
------	------

Random Test Cases:

Random 8-bit inputs to ensure design handles arbitrary data.

Includes both correct and deliberately incorrect hashes to test verification logic.

3. Key Features of the Testbench

Automated Verification:

Random test cases are generated using \$random.

Deliberate incorrect hashes test the failure detection mechanism.

Readability:

The formatted table allows for quick visual verification of all test cases.

Modularity:

The run_test task allows any test case to be applied easily, keeping the testbench clean and maintainable.

4. Summary

The testbench provides complete functional verification of the 8-bit SPN cipher.

It validates encryption, hash generation, and verification logic.

Both fixed and random tests ensure correctness and robustness.

Results are presented in a clear table format for documentation purposes.

5. Discussion of trade-offs (e.g., LUT vs. combinational logic, latency vs. complexity)

LUT vs. Combinational Logic

LUT (Look-Up Table): Used for S-box implementation.

Pros:

Very fast, single-cycle access.

Simple to implement in FPGA (most FPGAs have built-in LUTs).

Cons:

Consumes FPGA resources if the S-box is large.

Less flexible if we want to change the function dynamically.

Combinational Logic:

Implementing the S-box using logic gates instead of LUTs.

Pros:

Can reduce memory usage if FPGA LUTs are limited.

Can optimize for area if carefully designed.

Cons:

Slower propagation delay because of multiple logic levels.

Harder to implement for complex mappings.

Trade-off:

Using LUTs gives faster encryption but uses more FPGA resources.

Using combinational logic saves area but increases latency.

Latency vs. Complexity

Latency: Time taken to produce ciphertext from plaintext.

Complexity: Hardware resources used (LUTs, registers, wires).

Pipelining Trade-off: We have implemented a pipelined SPN cipher.

Pros:

Increases throughput (can encrypt a new byte every cycle).

Reduces critical path delay.

Cons:

Uses more flip-flops and registers.

Slightly more complex design.

But throughput is lower because the critical path is longer.

6. Any issues or common pitfalls encountered and how you addressed them

Issues and Common Pitfalls Encountered

During the design and verification of the 8-bit SPN cipher, several issues and potential pitfalls were identified. Below is a summary along with the solutions or workarounds implemented:

1. String Handling in Verilog

Issue:

Initially, a variable `result_str` was used to store "PASS" or "FAIL" and display it using %s in \$display.

Verilog does not support dynamic strings in synthesizable logic or in local variables for display like typical programming languages.

2. Incorrect PASS for All Cases

Issue:

Earlier, masking only the LSB of verified (`verified & 1'b1`) caused the comparison with expected results to always evaluate as true.

Solution:

Compared the full verified signal with the expected result using `===` operator:

```
if (verified === exp_result)
```

This ensures both correct and deliberately incorrect hashes are properly detected.

3. Random Test Case Failures

Issue:

Random inputs were occasionally causing unexpected verification results.

Root Cause:

The DUT had not fully propagated the hash or ciphertext before checking the verification flag.

Solution:

Introduced a #10 ns delay in the testbench after setting inputs to allow the DUT to complete computations.

4. S-box Implementation Pitfalls

Issue:

Misalignment in S-box tables can cause incorrect ciphertext generation.

Solution:

Carefully verified each S-box entry against the reference table.

Used consistent 8-bit hexadecimal notation (8'hXX) in case statements.

5. Combinational Loop Risk

Issue:

Designing multiple rounds in a fully combinational path can create long critical paths or accidental combinational loops.

Solution:

Checked signal dependencies and ensured all combinational assignments were stateless.

For larger block sizes, pipelining is recommended to reduce critical path delay.

7. Optional optimizations or additional features implemented.

While the primary goal of the project was to implement and verify a functional 8-bit SPN cipher, several optional enhancements and features were incorporated to improve robustness, usability, and extensibility of the design:

1. Random Test Case Generation

Feature:

The testbench includes random 8-bit input generation using \$random.

Benefit:

Allows thorough functional verification beyond fixed test vectors.

Helps uncover corner cases that might not be covered by manual inputs.

2. Deliberate Error Injection

Feature:

The testbench intentionally introduces incorrect expected hashes for some test cases.

Benefit:

Verifies the verification logic correctly identifies mismatches (FAIL).

Ensures the design reliably distinguishes between correct and incorrect outputs.

3. Modular Testbench with run_test Task

Feature:

The testbench uses a reusable run_test task for all test cases.

Benefit:

Simplifies adding new fixed or random tests.

Reduces code repetition and improves readability.

4. Formatted Console Output

Feature:

Results are displayed in a well-formatted table showing PLAINTEXT, CIPHERTEXT, HASH_VAL, EXPECTED_HASH, VERIFIED.

Benefit:

Provides clear, quick verification without analysing waveforms.

Useful for project reports and presentations.